Test Execution Report

18 test executions across 3 test classes: 10 passed, 8 failed.

Results by class

Test class	Total	Passed	Failed
WizardTest	9	6	3
ArchmageTest	5	1	4
SpellDispatchTest	4	3	1
Total	**18**	**10**	**8**

Detailed results

#	Test class	Test case	Expected result	Actual result	Status
1	WizardTest	restWhenExhaustedResetsStanceToIdle	Mana = 600; stance = IDLE	Mana restored, but stance = EXHAUSTED	Fail
2	WizardTest	spellSurgeWhileIdleThrowsWithoutChangingState	Throws InvalidStateException; mana and stance unchanged	Test passed	Pass
3	WizardTest	spellSurgePreservesManaInvariant(1)	Mana stays within 0–600; invalid investment rejected	Test passed	Pass
4	WizardTest	spellSurgePreservesManaInvariant(100)	Mana stays within 0–600; invalid investment rejected	Test passed	Pass
5	WizardTest	spellSurgePreservesManaInvariant(200)	Mana stays within 0–600; invalid investment rejected	Test passed	Pass
6	WizardTest	spellSurgePreservesManaInvariant(600)	Mana stays within 0–600; invalid investment rejected	Test passed	Pass
7	WizardTest	castSpellWithInsufficientManaPreservesMana	Throws ManaInvestedException; mana unchanged	No exception thrown	Fail
8	WizardTest	castSpellBelowRequiredLevelPreservesMana	Throws HeroLevelException; mana unchanged	Test passed	Pass
9	WizardTest	transferManaDeductsExactAmount	Sender = 500; recipient = 700	Sender = 599; recipient assertion not reported as failed	Fail
10	ArchmageTest	barrierDamageDoesNotMakeManaNegative	Mana ≥ 0 after damage	Mana = -1	Fail

#	Test class	Test case	Expected result	Actual result	Status
11	ArchmageTest	highPowerSurgeDoesNotAutomaticallyRest	Output = 360; mana = 480; stance = CHANNELING	Mana = 600; at least one assertion failed	**Fail**
12	ArchmageTest	highPowerSurgeDoesNotAutomaticallyRest2	Output = 360; mana = 480; stance = CHANNELING	Mana = 600; at least one assertion failed	**Fail**
13	ArchmageTest	highPowerSurgeRevealsSubclassStateDifference	Wizard output = 120; Archmage output = 360; both mana = 480; stances match	Archmage output = 180; Archmage mana = 600	**Fail**
14	ArchmageTest	wizardAndArchmagePreserveSharedStateBehavior	Wizard output = 1; Archmage output = 2; mana and stance match	Test passed	**Pass**
15	SpellDispatchTest	spellAndObjectReferencesSelectDifferentOverloads	Spell reference returns 40 and deducts 20 mana; Object reference returns 10 without mana deduction	Test passed	**Pass**
16	SpellDispatchTest	objectReferenceHoldingSpellUsesSpellHandling	Return value = 40	Return value = 10	**Fail**
17	SpellDispatchTest	safeWrapperDispatchesSpellCorrectly	Safe wrapper returns 40	Test passed	**Pass**
18	SpellDispatchTest	safeWrapperRejectsNonSpellObject	Throws IllegalArgumentException with message "Target must be a Spell"	Test passed	**Pass**

Notes

- spellSurgePreservesManaInvariant is a parameterized test with four inputs, so it counts as four test executions.
- In Test 9, the report identifies the sender's incorrect mana value (599 instead of 500). It does not show the recipient assertion failing, so the recipient likely reached the expected value of 700.
- For Test 13, the report explicitly shows the output mismatch (180 versus the expected 360) and the mana mismatch (600 versus 480).
- The Maven report confirms the failed assertions. Passing test details are based on the test code and the reported totals.

Complete Explanation of the JUnit Test Cases

The assignment tests a spellcasting system using JUnit 5. The goal is to expose subtle bugs involving object state, inheritance, mana calculations, and method overloading.

The Maven report shows 18 test executions: 10 passed and 8 failed. A passing test means the current implementation satisfies that particular test; a failing test reveals behavior that differs from what the test expects.

1. The eight assignment challenges

These are the challenge names and the test-class mapping from the assignment context.

Challenge	What it tests	Test class
1. Infinite Exhaustion Trap	rest() must restore mana and reset the stance from EXHAUSTED to IDLE.	WizardTest
2. Unchannelled Surge	Calling spellSurge() while idle must throw an exception without changing state.	WizardTest
3. Mana Boundary	Mana must remain between 0 and 600, including during spell surges and casts.	WizardTest
4. Archmage LSP Barrier Damage	Barrier damage must never make an Archmage's mana negative.	ArchmageTest
5. Yo-Yo Cascade	A high-power Archmage surge must not trigger unintended superclass state changes through rest().	ArchmageTest
6. Wizard/Archmage Polymorphic Behavioral Equivalence	Compare the behavior of Wizard and Archmage when using shared functionality.	ArchmageTest
7. Exact Mana Transfer	transferMana() must deduct exactly the requested amount from the sender.	WizardTest
8. Overload Dispatch	Java must call the correct overloaded method for Spell, Object, and wrapper references.	SpellDispatchTest

The sections below go through each of the 18 test executions: what it does, why the test exists, what the result demonstrates, and which challenge it addresses.

2. WizardTest.java — Challenges 1, 2, 3, and 7

This class contains 9 executions, including four runs of one parameterized test.

Test 1: restWhenExhaustedResetsStanceToIdle

FAIL *Challenge 1 — Infinite Exhaustion Trap*

What does it test?

It checks whether a Wizard can recover after becoming exhausted. Resting should restore mana to 600 and reset the stance to IDLE.

Expected vs. actual

Property	Expected	Actual
Mana after resting	600	600
Stance after resting	IDLE	EXHAUSTED

Why did it fail?

The rest() method restores the mana but does not reset the stance. The Wizard therefore has full mana while still marked as exhausted.

What does this demonstrate?

A method can update one part of an object's state correctly while leaving another part inconsistent. The test catches a state-transition bug.

The required behavior is conceptually:

```
manaStore = 600;
stance = Stance.IDLE;
```

The exact field names depend on the supplied implementation.

Test 2: spellSurgeWhileIdleThrowsWithoutChangingState

PASS *Challenge 2 — Unchannelled Surge*

What does it test?

It attempts to invoke spellSurge() while the Wizard is idle. The test expects an InvalidStateException and verifies that mana and stance remain unchanged.

Why is this important?

A spell surge should only be possible in an appropriate state. An invalid operation should be rejected rather than partially executed.

What does the result demonstrate?

The implementation correctly rejects the surge in the tested situation and preserves the object's state. This is an example of exception handling and state preservation.

Tests 3–6: spellSurgePreservesManaInvariant

PASS *Challenge 3 — Mana Boundary*

This is a parameterized test, meaning JUnit runs the same test logic with different input values.

Execution	Mana investment	Result
Test 3	1	PASS
Test 4	100	PASS
Test 5	200	PASS
Test 6	600	PASS

What does it test?

It checks that spell surges respect the mana boundaries:

$$0 \leq manaStore \leq 600$$

The test also checks that an investment of 601, which exceeds the allowed amount, throws ManaInvestedException without changing mana.

Why test multiple values?

A method might work for a small value but fail near a boundary. Testing different investments helps reveal boundary-related bugs. For example:

- A small investment tests ordinary behavior.
- An investment of 600 tests the upper boundary.
- An investment of 601 tests invalid input.

What does the result demonstrate?

All four recorded executions passed. The tested inputs satisfy the assertions, but this does not prove that every possible input is safe.

Test 7: castSpellWithInsufficientManaPreservesMana

FAIL *Challenge 3 — Mana Boundary*

What does it test?

The Wizard has 600 mana, but the spell costs 601 mana. The test expects ManaInvestedException because the Wizard cannot afford the spell.

Expected vs. actual

Property	Expected	Actual
Exception	ManaInvestedException	No exception
Mana	Remains unchanged	The test's exception expectation fails

Why did it fail?

According to the supplied implementation notes, castSpell(Spell) does not perform the required mana check before casting.

What does this demonstrate?

The implementation permits a spell to proceed even when its cost exceeds the available mana. This violates the mana-boundary rule. A robust implementation should validate the available mana before deducting any cost or executing the spell.

Test 8: castSpellBelowRequiredLevelPreservesMana

PASS *Challenge 3 — Mana Boundary*

What does it test?

It attempts to cast a spell for which the Wizard's level is too low. The expected result is HeroLevelException, with mana unchanged.

Why does this matter?

Level validation is another precondition for casting. If a spell cannot be cast, the attempt should not consume mana.

What does the result demonstrate?

The level restriction is enforced in the tested case, and mana remains unchanged. This shows that the level-validation path works even though the separate insufficient-mana path fails.

Test 9: transferManaDeductsExactAmount

FAIL *Challenge 7 — Exact Mana Transfer*

What does it test?

It transfers mana between two Wizards and checks whether the sender loses exactly the amount transferred.

Expected vs. actual

Property	Expected	Actual
Sender's mana	500	599
Recipient's mana	700	No recipient assertion failure reported

Why did it fail?

The supplied implementation notes identify a bug in transferMana(): it subtracts priorityTier instead of the requested transfer amount. The sender therefore loses 1 mana rather than the expected 100 in this test.

What does this demonstrate?

The method is using the wrong variable for its calculation. This is a business-logic bug: the method executes, but its arithmetic does not match the requested operation.

3. ArchmageTest.java — Challenges 4, 5, and 6

This class contains 5 executions: 1 passed and 4 failed.

Test 10: barrierDamageDoesNotMakeManaNegative

FAIL *Challenge 4 — Archmage LSP Barrier Damage*

What does it test?

It checks that barrier damage cannot push the Archmage's mana below zero.

Expected vs. actual

Property	Expected	Actual
Mana after barrier damage	At least 0	-1

Why did it fail?

The implementation subtracts the damage directly from manaStore without ensuring that the result remains nonnegative.

What does this demonstrate?

The Archmage violates the expected mana invariant. A subclass must preserve the important guarantees expected of its parent type; otherwise, code that works with a regular Wizard may behave incorrectly when given an Archmage. The damage calculation needs a guard or another rule that prevents negative mana.

Test 11: highPowerSurgeDoesNotAutomaticallyRest

FAIL *Challenge 5 — Yo-Yo Cascade*

What does it test?

It tests whether a high-power Archmage surge can complete without unexpectedly triggering a rest operation. The test expects an output of 360, mana of 480, and a CHANNELING stance.

Observed failure

- Expected mana: 480

- Actual mana: 600

The Maven report identifies the mana assertion as a failure.

Why did it fail?

The implementation's high-power surge path calls rest(). That restores mana to 600, undoing the expected mana state for the surge.

What does this demonstrate?

One method can cause an unexpected state change by calling another method. This is the Yo-Yo Cascade problem: the surge changes state, then the automatic rest changes it again. The intended behavior is to complete the surge while preserving its expected mana and stance, rather than automatically restoring mana.

Test 12: highPowerSurgeDoesNotAutomaticallyRest2

FAIL *Challenge 5 — Yo-Yo Cascade*

What does it test?

This is another test of the high-power surge behavior, checking that the Archmage does not automatically rest and reset its mana.

Observed failure

- Expected mana: 480
- Actual mana: 600

The test fails on the mana assertion, just like Test 11.

Why have two tests for the same challenge?

They can exercise the behavior under different test setups or execution conditions. Multiple tests make it less likely that a state-transition bug will go unnoticed in one particular scenario.

What does this demonstrate?

The unwanted rest behavior is reproducible, rather than an isolated failure.

Test 13: highPowerSurgeRevealsSubclassStateDifference

FAIL *Challenge 5 — Yo-Yo Cascade*

What does it test?

It compares the high-power surge behavior of a Wizard and an Archmage to reveal whether the subclass changes state or output unexpectedly.

Expected vs. actual for the Archmage

Property	Expected	Actual
Surge output	360	180
Mana	480	600

Why did it fail?

The Archmage's high-power surge behavior does not match the expected result. The implementation's automatic-rest path is consistent with the unexpected mana restoration, while the output also differs from the expected value.

What does this demonstrate?

The subclass's behavior needs to be examined independently of the base class. Inheritance can introduce unexpected behavior when a subclass overrides a method or calls inherited functionality with different effects.

This test provides evidence of a subclass-specific behavioral problem. The exact cause of the output discrepancy should be confirmed by tracing the surge calculation and its method calls.

Test 14: wizardAndArchmagePreserveSharedStateBehavior

PASS *Challenge 6 — Polymorphic Behavioral Equivalence*

What does it test?

It compares a Wizard and an Archmage using shared behavior and checks their output, mana, and stance.

What does the result demonstrate?

Both objects satisfy the assertions for this particular scenario. The test shows that their shared-state behavior is consistent under the tested conditions.

Important distinction: passing this test does not mean the classes are equivalent in every situation. The high-power surge tests already show that their behavior can diverge in other scenarios.

4. SpellDispatchTest.java — Challenge 8

This class contains 4 executions: 3 passed and 1 failed.

Test 15: spellAndObjectReferencesSelectDifferentOverloads

PASS *Challenge 8 — Overload Dispatch*

What does it test?

It invokes overloaded methods through two different reference types:

- A Spell reference calls the Spell overload, returning 40 and deducting 20 mana.
- An Object reference calls the Object overload, returning 10 without deducting mana.

Why does the result differ?

Java selects an overloaded method using the declared type of the reference at compile time, not merely the runtime class of the object. For example:

```
Spell spell = new Spell(/* arguments */);
Object object = spell;
```

A call using spell can select castSpell(Spell), while a call using object can select castSpell(Object).

What does this demonstrate?

The test confirms Java's compile-time overload resolution. The two calls behave differently because they select different overloads.

Test 16: objectReferenceHoldingSpellUsesSpellHandling

FAIL *Challenge 8 — Overload Dispatch*

What does it test?

It stores a Spell object in a variable declared as Object and expects the spell-specific handling to occur.

Expected vs. actual

Property	Expected	Actual
Result	40	10

Why did it fail?

The call selects the Object overload because the reference is declared as Object. Java does not automatically choose the more specific overload based on the object's runtime type.

What does this demonstrate?

This test exposes a common misunderstanding about method overloading. If the program requires spell-specific behavior even when the reference is typed as Object, it needs explicit type checking, casting, or a wrapper method that dispatches appropriately.

Test 17: safeWrapperDispatchesSpellCorrectly

PASS *Challenge 8 — Overload Dispatch*

What does it test?

It calls a wrapper that routes a valid Spell object to the correct spell-handling logic.

What does the result demonstrate?

The wrapper successfully produces the expected result of 40. This shows that explicit dispatch can avoid the overload-selection issue encountered when calling the method directly through an Object reference.

Test 18: safeWrapperRejectsNonSpellObject

PASS *Challenge 8 — Overload Dispatch*

What does it test?

It passes an object that is not a Spell to the safe wrapper. The test expects:

```
IllegalArgumentException
```

with the message:

```
Target must be a Spell
```

What does the result demonstrate?

The wrapper validates the input and rejects the wrong type with the expected exception and message. This is a useful example of defensive programming: invalid inputs are handled deliberately rather than being processed as if they were valid spells.

5. Final summary: what the tests reveal

Challenge	Main finding	Result
1. Infinite Exhaustion Trap	Rest restores mana but fails to reset the stance.	**Failed**
2. Unchannelled Surge	Idle surges are rejected without changing state.	**Passed**
3. Mana Boundary	Tested surge investments pass, but insufficient-mana casting is not rejected.	**Mixed**
4. Archmage LSP Barrier Damage	Barrier damage can reduce mana to -1.	**Failed**
5. Yo-Yo Cascade	High-power surges unexpectedly restore mana and produce an unexpected output.	**Failed**
6. Polymorphic Behavioral Equivalence	Shared behavior matches in the tested scenario.	**Passed**
7. Exact Mana Transfer	The sender loses the wrong amount of mana.	**Failed**
8. Overload Dispatch	Direct Object overload selection differs from spell-specific dispatch; the wrapper works.	**Mixed**